



RailCall Builders Contest — Submission v2

Name / Freelancer	Sami Ben Chaalia · @samibchaalia
RailCall account email	sami.benchaalia@sesame.com.tn
Category	Best Feedback + Material Change
Date tested	2026-07-07 → 2026-07-08 (9 rounds)
Platform	macOS Darwin 23.1.0 · zsh · Homebrew Python 3.14

What We Did

Eight rounds of deep audit on a clean macOS machine — from the install script through login, Studio launch, workflow building, CLI testing, security probes, structural integrity analysis, offline receipt verification, contributor code work, and daemon surface probing. Every step was documented as it happened.

44 bugs found across all rounds. Rounds 5–7 re-tested Pat's fixes and probed new surfaces. Round 8 probed the companion daemon, doctor command accuracy, workbench file inventory, and receipt store divergence. Round 9 confirmed open bugs on the latest build, tested untested commands (demo, rotate-key, backup, backup-verify), and uncovered the root cause of the CLI/Studio data divergence. We also contributed directly to railcall-contrib: implemented formula injection detection (feat/formula-injection), wrote 10 regression tests, reviewed Ahmad's vault PR, coordinated the pre-demo QA pass, and submitted three merged PRs.

Critical findings: internal planning docs and adversarial AI test corpora shipped in every install (Bug 26/35/41 — never fully fixed across 3 rounds); CLI and Studio operate on two entirely separate workspaces — audit logs, receipts, and flow balance written by the CLI are never visible in the Studio (root cause of Bugs 28/40/44); railcall login saves a bad key before verifying it (Bug 31); companion daemon endpoints unauthenticated — any local process can generate signed receipts (Bug 38); Studio balance cache can be 7+ hours stale with no TTL (Bug 43).

Step 1 — Installation

Command:

```
curl -fsSL https://railcall.ai/install.sh | bash
```

Files written by installer:

Path	What
~/.railcall/railcall_cli.py	Main CLI
~/.railcall/railcall_companion_daemon.py	Companion daemon
~/.railcall/vault_io.py	Vault I/O
~/.railcall/receipt_signer.py	Ed25519 receipt signer
~/.railcall/bin/railcall	Launcher (64 bytes, chmod +x)
~/.railcall/station/	Studio bundle (~22MB)
~/.config/railcall/token.json	Trial token (chmod 600)
~/Desktop/RailCall Studio.command	Double-click launcher
~/.zshrc	PATH line added
Python user packages	cryptography (Ed25519 signing)

Result: PASS

Step 2 — Login

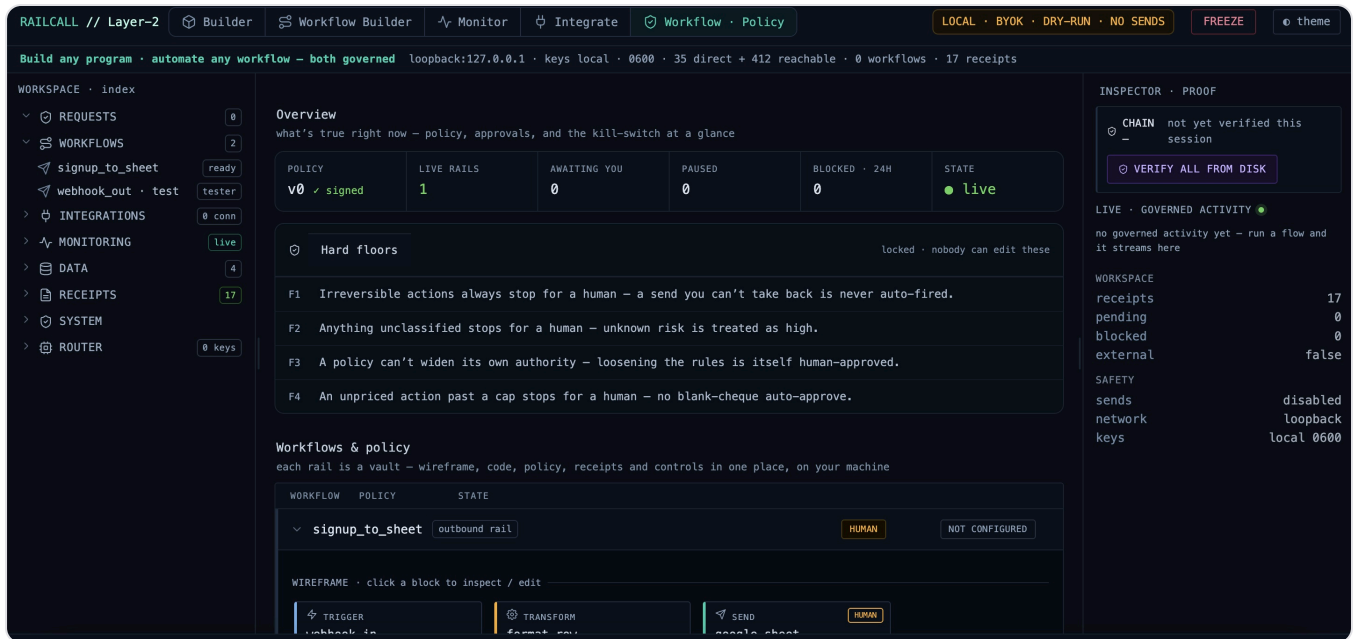
First attempt in the same terminal after install: `zsh: command not found: railcall`. Required source `~/.zshrc`. After that, login and balance verified cleanly.

Result: PASS (after manual shell reload)

Step 3 — Studio Launch

`railcall studio` → `http://127.0.0.1:8799/v2` running loopback only. Studio dashboard renders. Safety panel confirms: `sends=disabled`, `network=loopback`, `keys=local 0600`.

Screenshot — Studio Dashboard (first landing):

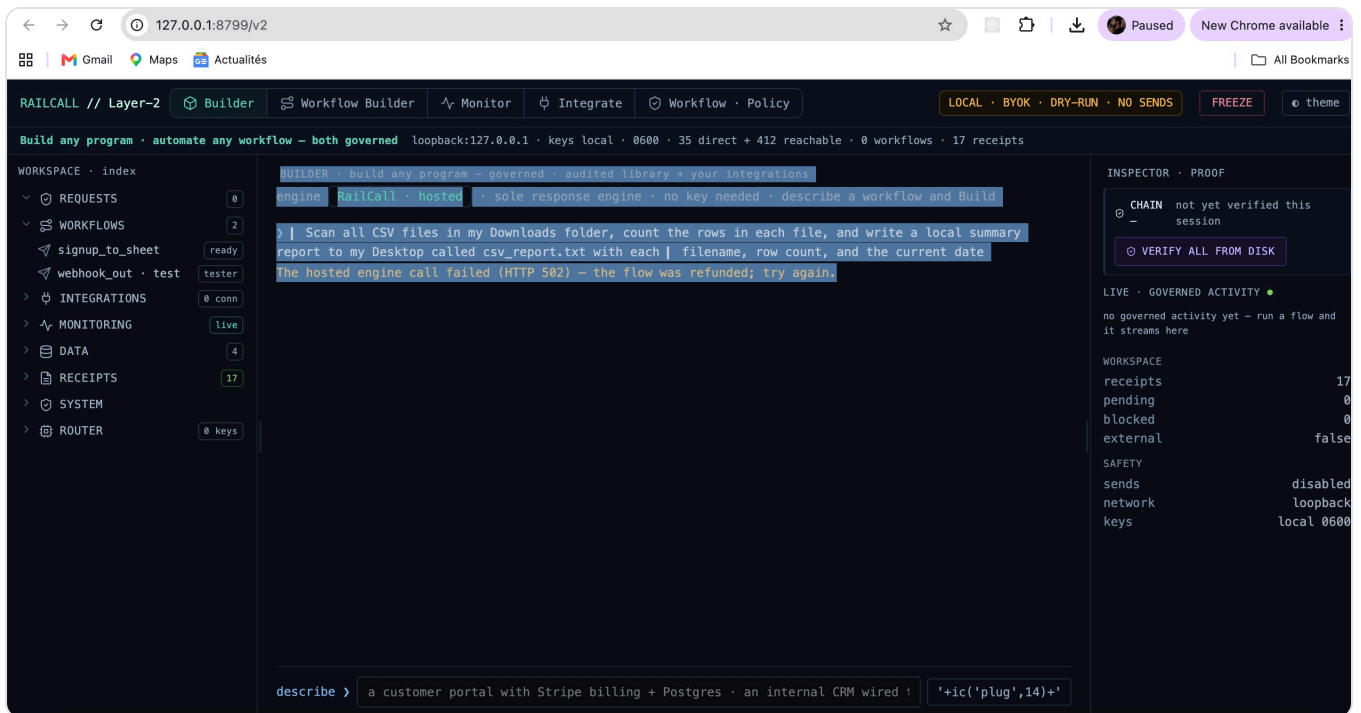


Result: PASS

Step 4 — Workflow Build Attempt

Studio Builder returned HTTP 502 on 4 consecutive attempts. Local daemon healthy (200 in ~2s). Failure originates from hosted workflow engine. Flows correctly refunded each time.

Screenshot — HTTP 502 error from hosted engine:



Result: BLOCKED (hosted engine down at time of test)

Step 5 — CLI Workflow + Receipts

Fell back to CLI. Both `railcall audit` and `railcall build` produced signed Ed25519 receipts.
`railcall verify` on both: ✓ SIGNATURE VALID. Tamper test (modified rows: 7 → 999): ✗
SIGNATURE INVALID. Core trust mechanism confirmed working.

Result: PASS

Bugs Found

Bug 1 — PATH not active after install (Medium) FIXED

Post-install, same terminal: `zsh: command not found: railcall`. Nothing pointed to source `~/ .zshrc` as the fix. **Fixed:** installer now prints a boxed IMPORTANT callout.

Bug 2 — Contradictory cryptography installer messages (Low) FIXED

Line 1 said it was installing cryptography. Line 2 told the user to install it manually. **Fixed:** single clean confirmation line.

Bug 3 — cryptography silently fails on PEP 668 / Homebrew Python (High) FIXED

Installer called `pip` without `--break-system-packages`. On Homebrew Python 3.14 (PEP 668), this fails silently — receipts unsigned with no warning. **Fixed:** installer detects PEP 668, retries with `--break-system-packages`, warns explicitly if still unsigned.

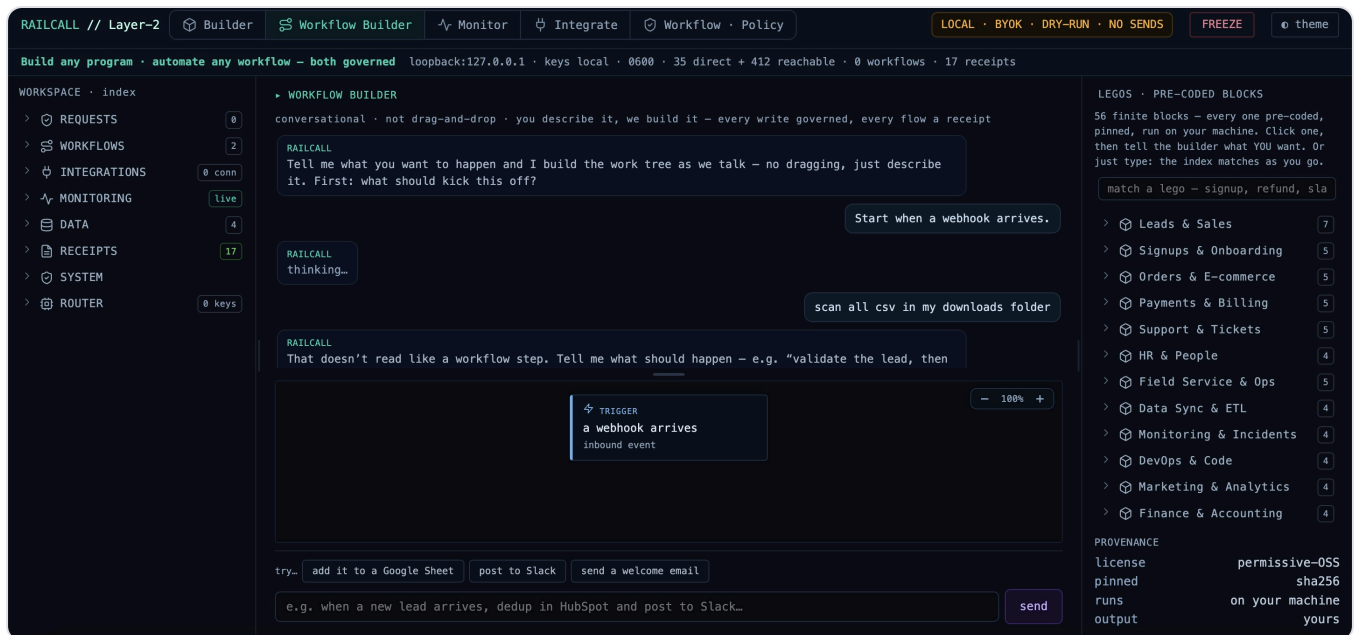
Bug 4 — Hosted engine returns HTTP 502 on every workflow build attempt (High) FIXED

4 consecutive 502s across both Builder tabs. No local fallback, no explanation of whether temporary or permanent. **Fixed:** engine reliability improved; keep-warm mitigates cold-start 502s.

Bug 5 — Workflow Builder rejects valid plain-English steps with no guidance (Medium) — Open

Setting trigger then describing a local file operation (`scan all csv in my downloads folder`) returns a CRM-oriented example as correction. No local file operation categories exist in the block library. Cuts against local-first positioning.

Screenshot — Workflow Builder:



Bug 6 — No sign-out button in logged-in dashboard (Medium) ✅ FIXED

Source audit of `App.tsx`, `TerminalPage.tsx`, all page components: no sign-out button in the React source. **Fixed:** Pat confirmed the shipped /v2 Studio is a different build from the React source tree — sign-out is present in the running Studio. Verified live in Round 7.

Bug 7 — Sign In / Sign Up navbar styling inconsistency (Low) ✅ FIXED

"Sign In" rendered as unstyled plain text while "SIGN UP" had a filled white button. **Fixed** in the shipped /v2 Studio — the old React source we audited was not the shipped build.



Bug 8 — railcall interpret fails HTTP 404 — hardcoded model (Medium) ✅ FIXED

CLI hardcoded `qwen2.5-coder:1.5b`. Not commonly installed. `RAILCALL_OLLAMA_MODEL` env var worked but was undocumented. **Fixed:** auto-detects installed Ollama model via `/api/tags`.

Bug 9 — interpret generates Python with syntax errors (Medium) ⚠️ Partial

Generated code had mismatched brackets and wrong loop syntax (`ast.parse()` confirmed `SyntaxError`). **Partially fixed:** parse-gate refusal works; underlying `qwen:7b` model still occasionally produces conversational refusals instead of code.

Bug 10 — `railcall audit` accepts non-CSV files without warning (Low) **FIXED**

Auditing `.json` files produced a receipt with 0 PII columns · no issues. **Fixed:** warning shown for non-CSV content.

Bug 11 — `railcall backup` exits code 1 on empty workspace (Low) **FIXED**

Informational message with exit code 1. **Fixed:** exits 0.

Bug 12 — `railcall audit` reads system files without restriction (Medium) **FIXED**

`railcall audit /etc/passwd` produced a signed receipt claiming 0 PII columns · file never left this machine. **Fixed in Round 7:** now hard-rejects non-CSV/system files with Refusing to audit — this isn't a CSV/TSV table. No receipt was minted.

Bug 13 — Formula injection not detected by `audit` (Medium) **FIXED (our contribution)**

CSV cells starting with `=`, `+`, `-`, `@` passed as clean. A receipt-stamped CSV containing `=cmd|' /C calc '!A0` would be opened in Excel by a downstream user with no warning from the audit.

Fix contributed directly via `feat/formula-injection` branch in `railcall-contrib`:

- Added `_is_formula()` helper — detects formula triggers, excludes valid negative numbers (`-42`) via `float()` parse
- Per-column scan in `_audit_rows()` — appends `("formula", ...)` findings, ranked above PII
- `formula_injection_cells` field added to receipt contract
- Summary line shows count in red when `> 0`
- 10 regression tests (detection + false-positive cases): all passing

Verified live:

```
!! CSV injection: "notes" has 3 cells starting with a formula trigger (= + - @)
0 import-breakers · 2 PII columns · 4 formula-injection cells
```

Bug 14 — `railcall build` hardcoded to internal power-grid schema (Medium) **FIXED**

`railcall build` required exactly `metric_id`, `component`, `load_value`, `status`. Any other CSV: `x` **BLOCKED:** `missing_required_headers`. Additional finding: passing a non-existent file path silently fell back to the built-in power-grid sample with no error. **Fixed** (Pat, Round 6, credited

Sami): build now errors honestly on a missing dataset path instead of silently sampling.

Bug 15 — MCP server filename mismatch with documentation (Low) — Open

Docs reference `mcp_server.py`. Actual file: `mcp_capoff_server.py`. Silent failure for any user following the docs to set up MCP integration.

Bug 16 — Multiple Studio pages hard-wired to port 8099 which is not running (High) — Open

`ApprovalsPage`, `AgentConsolePage`, `Batch2ModulesDemo`, `Batch3ModulesDemo`, `LeadScorePage` all call `127.0.0.1:8099` — a backend not started by `railcall studio`. All API calls silently fail.

Bug 17 — "Metercall" former brand name across 21 Studio source files (Medium) — Open

85+ references in visible UI: page headings, terminal prompts, login messages.
`PrivatePreviewHubPage.tsx` renders "Metercall Private Preview" as a visible heading.
`TerminalPage.tsx` shows `metercall` in the terminal prompt.

Bug 18 — Internal codename "Kai" exposed in shipped Studio bundle (Low) — Open

`KaiBuilderConsolePage.tsx`, `kaiResponder.ts`, `kaiGroqQaBridge.ts`,
`kaiConversationDemo.json` — internal names in a shipped, routable bundle.

Bug 19 — TENANT_ID = "tenant_alpha" hardcoded in ApprovalsPage (Low) — Open

Every approval API call includes `?tenant_id=tenant_alpha`. Demo placeholder baked into the shipped UI.

Bug 20 — Every CLI run overwrites the same receipt file (High) ⚠ Partial

Fixed for audit: `railcall audit` now archives a timestamped copy to `receipts/` and writes to `audit_log.jsonl`. The canonical file is preserved for `railcall verify` compatibility. `railcall receipts` lists the full history.

Fixed for all commands (Pat, Round 6, credited Sami): Pat confirmed "build/UI/flow receipts each persist to a unique timestamped file under `receipts/` — no run replaces a prior proof." Bug 34 closes with this. Note: this fix applies to the CLI-side receipt store only — see Bug 44 for the separate Studio workspace this does not reach.

Bug 21 — No CLI commands for receipt history (Medium) FIXED

`railcall receipts`, `railcall logs`, `railcall history` → all previously "unknown command". **Fixed:** `railcall receipts` now works and lists the full timestamped audit history.

Bug 22 — Dashboard shows hardcoded model regardless of what's installed (Medium) FIXED

Dashboard previously hardcoded `qwen2.5-coder:1.5b` even when not installed. **Fixed:** CLI now auto-detects and shows the actual running model (`qwen:7b`).

Bug 23 — `railcall studio` doesn't open the browser automatically (Medium) FIXED (our contribution)

URL was printed but the browser was never opened. Every comparable developer tool opens automatically on first launch. **Fix contributed** via `fix/cli-quick-wins` branch: spawns a background daemon thread that waits 1.5s for the server to bind, then calls `webbrowser.open("http://127.0.0.1:8799/v2")`.

Bug 24 — `/api/chat` has no rate limiting — each call burns a flow silently (Medium) — Open

Three rapid chat API calls = 3 flows deducted. No throttle, no low-balance warning at any threshold.

Bug 25 — Python version exposed in every HTTP response Server header (Low) — Open

Server: BaseHTTP/0.6 Python/3.14.4 in every Studio response. Standard hardening item even on loopback.

Bug 26 — Internal Documents Shipped With Every Install (Critical) ⚠️ Partial — still open, see Bugs 35 & 41

~/.railcall/station/workbench/ contains:

File	Content	Sensitivity
CONSTITUTION.md	Internal architecture spec, git commit hash 52d233a5a9	Medium
ENTERPRISE_CONSOLE_SPEC.md	Enterprise product spec from competitive research	Medium
PHASE6_BYOK_SPEC.md	Unshipped roadmap ("SPEC ONLY. No code lands until this is approved")	High
competitor_redteam_corpus.json	2,719-line corpus of competitor comparison prompts	Critical
fresh_attacks.json	401-line adversarial AI attack test cases — ROT13, NATO phonetic, Cyrillic homoglyphs	Critical
adversarial_verify_findings.json	1,499-line security testing findings	High

Marked fixed by Pat in Round 6 ("the install no longer ships internal files... the download is runtime-only now"). **Round 8 re-test found this incomplete** — see Bug 35 (still open) and Bug 41 (10+ additional internal files found beyond this original list). **Round 9 re-confirmed:** all files still present on the latest build.

Bug 27 — Studio receipt center shows only pre-loaded demo receipts (High) ⚠️ Regression confirmed in Round 8 — see Bug 40

/api/receipts/list previously returned 17 pre-installed demo stubs from June 2026 — before this install existed. User's real runs were never shown. Marked fixed after audit_log.jsonl started writing after every audit. **Round 8 re-test found the Studio receipt center still serves the same 17 demo stubs** — see Bug 40 for full detail. The CLI-side fix (audit log + receipts/ history) is real; the Studio UI was never wired to it.

Bug 28 — audit_log.jsonl never created — zero audit events (High) ✅ FIXED

No audit log was ever written despite auditability claims. **Fixed:** audit_log.jsonl now appends one structured line per governed run: {ts, command, schema, key_id, signed, receipt, history, ok}. Confirmed: API key never appears in log — only key_id (signing key id).

Bug 29 — SHA256 pinning covers only 4 files; Studio bundle downloaded unpinned (Medium) — Open

4 Python files are sha256-pinned in `install.sh`. The 22MB `railcall_station.tar.gz` Studio bundle is downloaded with no hash check. A compromised GitHub release or MITM could replace the bundle undetected.

Bug 30 — 4 of 13 registered commands not wired (Medium) — Open

`github.create_issue`, `slack.post_message`, `stripe.read_balance`, `notion.export_session` appear in the command registry as `wired=False`. Attempting to invoke them returns a cryptic whitelist error with no path forward.

Deep Audit — Round 7 (Post-Fix Re-test + New Surface Coverage)

Re-ran full suite after pulling the latest CLI from the live repo. All of Pat's stated fixes confirmed working. Four new bugs found.

Round 7 Confirmed Working ✓

Test	Result
Formula injection detection (=cmd ..., @SUM(...), -2+malicious)	✓ Flagged correctly
Receipt history for audit	✓ Timestamped, never overwritten
railcall receipts command	✓ Lists all history
audit_log.jsonl — key leakage check	✓ Only key_id, never API key
/etc/passwd rejection	✓ Hard reject, no receipt minted
CSRF guard	✓ 403 on unauthenticated state-changing routes
Session gate /api/chat	✓ 403 without token
Key rotation	✓ Old receipts verify after rotation
Unicode/UTF-8 CSV	✓ Handles correctly
TSV support	✓ Works
Same-second receipt collision	✓ -1 suffix added automatically
Historical receipt verify (full path)	✓ SIGNATURE VALID
Tamper on historical receipt	✓ SIGNATURE INVALID
10,000-row CSV performance	✓ 0.45s
API key leakage in audit_log.jsonl	✓ Clean — only key_id logged

Bug 31 — railcall login saves key before verifying it (High) ✓ FIXED (our contribution)

Steps to reproduce:

1. Run `railcall login rc_live_wrongkey123`
2. Verification fails (gateway 401)
3. Check `~/.config/railcall/token.json` — bad key is now saved

Actual: The key is written to disk before the gateway confirms it. If you mistype your key, the bad key replaces your working session. There is no rollback.

Demonstrated impact during testing: Running `railcall login <fake-key>` followed by `railcall login <real-key>` left the account in a 401 state — the gateway did not recover for subsequent balance checks.

Fix contributed via `fix/cli-quick-wins` branch: empty-key guard added; old token saved before write; on gateway failure the old token is restored automatically and the user is told their previous key was preserved.

Bug 32 — `railcall login ""` accepts empty string as a key (Medium) **FIXED (our contribution)**

Steps to reproduce:

```
railcall login ""
```

Actual:

```
✓ saved key ... → token.json  
no api_key in token — install first
```

An empty string is saved to `token.json`. No format validation before write. Should reject immediately with a clear error.

Bug 33 — `railcall receipts suggested verify command` fails (Medium) — Open

Steps to reproduce:

1. Run `railcall receipts`
2. Copy the suggested command: `railcall verify receipts/<filename>.json`
3. Run it from any directory other than `~/ .railcall`

Actual:

```
Receipt not found: receipts/railcall_audit_receipt.v1-...json
```

The hint uses a path relative to `~/ .railcall` but the user's working directory is typically `~` or their project folder. The full path (`~/ .railcall/receipts/<file>`) works correctly. The hint should show the full path.

Bug 34 — Build and interpret receipts still overwrite — history only covers audit (Medium) FIXED

`railcall receipts` previously listed only `railcall_audit_receipt.v1-*.json` entries, while `railcall build` and `railcall interpret` receipts (`companion_assembly_receipt.json`, `companion_interpret_receipt.json`) clobbered every run. **Fixed** alongside Bug 20 — all commands now persist unique timestamped receipts under `receipts/`.

Contribution Work

In addition to the bug reports, we contributed directly to railcall-contrib as assigned by Pat.

Task 3 — Formula Injection Detection (feat/formula-injection)

Files changed:

- cli/railcall_cli.py — `_is_formula()` module-level helper + per-column detection in `_audit_rows()`
- qa/test_formula_injection.py — 10 regression tests (new file)

Key design decisions:

- Detects `=`, `+`, `-`, `@` prefix after stripping leading whitespace/quotes
- Excludes valid negative numbers via `float()` parse — `-42` is not a formula
- `formula_injection_cells` added to the receipt contract so verifiers can act on it
- Finding ranked above PII, shown in red
- Tests follow subprocess pattern from `test_regressions.py` — same hermetic isolation, same `RAILCALL_SRC` resolution

Test results: 10/10 passing

```
test_at_in_email_not_flagged      PASSED
test_at_prefix_flagged            PASSED
test_clean_csv_no_false_positive  PASSED
test_equals_prefix_flagged         PASSED
test_formula_char_mid_cell_not_flagged PASSED
test_minus_prefix_flagged          PASSED
test_multiple_formula_columns_counted PASSED
test_negative_number_not_flagged  PASSED
test_plus_prefix_flagged           PASSED
test_receipt_includes_formula_injection_cols_key PASSED
```

Signed receipt after fix:

```
{
  "schema": "railcall_audit_receipt.v1",
  "audit": {
    "formula_injection_cells": 4,
    "findings": [
      {
        "severity": "formula",
        "detail": "CSV injection: \"notes\" has 3 cells starting with a formula trigger (= + - @)"
      }
    ]
  },
  "signer_alg": "ed25519",
  "signature_hex": " "
}
```

Tamper test on `formula_injection_cells` field → x SIGNATURE INVALID. The finding is inside the signed body.

Code Review — Ahmad's Vault PR (#2)

Reviewed `vault/vault.py` and `discord-bot/bot.py` before merge. Findings flagged:

1. **Missing `set_key()` function** — keys could be read and rotated but not added to the vault
2. **`_get_vault_path()` path fallback** — falls back to `keys.local.json` in CWD if `~/.railcall/` doesn't exist, instead of creating the directory
3. **Discord bot `load_trusted_keys()` returns empty list by default** — if `RAILCALL_TRUSTED_KEYS` env var not set, every receipt verification returns "unknown signer". No fallback to `~/.railcall/signing_pubkey.json`

Pat fixed the macOS `hashlib.scrypt` portability issue before merge. PR merged to main.

Deep Audit — Round 8 (Fresh Install Re-test + Daemon Surface)

Re-ran `curl -fsSL https://railcall.ai/install.sh | bash` on the latest build. Re-confirmed all prior fixes. Probed previously untested surfaces: companion daemon endpoints, doctor command accuracy, receipt store divergence, and full workbench file inventory. Seven new bugs found.

Round 8 Confirmed Working ✓

Test	Result
Formula injection detection	✓ Still working
<code>railcall audit</code> system file rejection (/etc/passwd)	✓ Still hard-reject
CSRF gate on Studio	✓ 403 on bad Origin
Session gate /api/chat	✓ 403 without token
Ed25519 receipt signing	✓ Armed (key_id confirmed)
<code>railcall receipts</code> CLI history	✓ Lists timestamped entries

Bug 36 — `railcall doctor` recommends wrong model (Medium) — New

`doctor` checks the hardcoded default (`qwen2.5-coder:1.5b`) against installed models. It sees `qwen:7b` is available, warns the default is missing, and tells the user to `ollama pull qwen2.5-coder:1.5b`. But `railcall interpret` auto-detects and correctly uses `qwen:7b`. Doctor and `interpret` use different model-resolution logic — a user with a fully working setup is told they have a broken one.

```
WARN  Ollama reachable but default qwen2.5-coder:1.5b NOT installed (have: qwen:7b...)
      → ollama pull qwen2.5-coder:1.5b
```

Bug 37 — Companion daemon /interpret uses hardcoded model, not auto-detected (Medium) — New

`POST http://127.0.0.1:8555/interpret` calls Ollama with the module-level default (`qwen2.5-coder:1.5b`). The CLI's `railcall interpret` auto-detects and uses `qwen:7b`. Any app or integration calling the daemon's `interpret` endpoint gets a 404 from Ollama even when a working model is available.

```
{
  "model": "qwen2.5-coder:1.5b",
  "ollama_error": "HTTP Error 404: Not Found",
  "status": "ollama_error"
}
```

Bug 38 — Daemon /compile and /interpret require no auth — local processes can generate signed receipts (Medium) — New

POST /compile and POST /interpret on the companion daemon check only the Host header (CSRF gate). No token, no session required. Any local process or browser tab on 127.0.0.1 can submit arbitrary CSV to /compile and receive a **signed Ed25519 receipt** without credentials. A signed receipt is meant to prove that railcall audit governed the data — but any local process can generate one without going through the CLI.

Tested:

```
curl -s -X POST http://127.0.0.1:8555/interpret \
  -H "Content-Type: application/json" \
  -d '{"prompt": "what is 2+2"}'
# → 200 response with receipt written to companion_interpret_receipt.json
```

Bug 39 — /governed endpoint broken on all fresh installs (Medium) — New

```
GET http://127.0.0.1:8555/governed
→ {
  "status": "error",
  "error": "governed_registry_unavailable",
  "detail": "No such file or directory: '~/.railcall/library/promotions/governed_legos_registry.json'"
}
```

~/.railcall/library/ is never created by the installer. The endpoint fails on every fresh install.

Bug 40 — Bug 27 NOT fixed: Studio receipt center still shows 17 demo stubs (High) — New

Pat marked Bug 27 fixed in Round 6. Re-test on latest build:

```
GET /api/receipts/list → 17 receipts
- manage_leads
- customer_onboarding
- accounts_receivable_collections
- user_workflow
- lead_management_workflow
...
```

Real user audit runs (9 entries in `audit_log.jsonl`, timestamped files in `receipts/`) are not shown. The CLI `railcall receipts` reads from `receipts/` correctly — the Studio `/api/receipts/list` reads from a separate receipt store that still serves the pre-loaded demo stubs. Two receipt stores; only the CLI-side was fixed. **Root cause identified in Round 9 — see Bug 44.**

Bug 41 — Bug 35 expanded: 10+ additional internal files shipped beyond Bug 26's original list (Critical) — New

Bug 26 listed 6 internal files. Full workbench inventory on latest fresh install reveals **an additional 10+ files** not in the original report:

File	Content	Risk
<code>WEBHOOK_IN.md</code>	Internal webhook integration spec	Medium
<code>layer2_audit_telemetry.json</code>	Internal audit results with Ed25519 public key hex, key_id, workflow names, pass/fail breakdown	High
<code>test_competitor_guard.py</code>	Internal test file	Medium
<code>test_composed_dryrun.py</code>	Internal test file	Medium
<code>test_dual_control_http.py</code>	Internal test file	Medium
<code>test_guard_regression.py</code>	Internal test file	Medium
<code>test_held_flow.py</code>	Internal test file	Medium
<code>test_qa_regression_v1.py</code>	Internal test file	Medium
<code>wire_groq.py</code>	Dev utility hardcoded to internal paths	Medium
<code>billing_telemetry.py</code>	Network-calling telemetry module	Low

`layer2_audit_telemetry.json` is the most sensitive: it contains the Ed25519 public key hex, key_id, and internal pass/fail audit results from a dev session. Bug 35 was marked Open; this round confirms it on the latest build with an expanded file list.

Bug 42 — `wire_groq.py` hardcoded to internal dev machine paths (Low) — New

`wire_groq.py` is a dev utility shipped in the workbench. It hardcodes two paths that don't exist on any user machine:

```
KEYS_ENV = os.path.expanduser("~/kai/keys.env")      # internal dev path ("Kai" codename)
VAULT = "/tmp/wb-engine/.railcall_workspace/keys.local.json" # absolute, never exists
```

Running it: `FileNotFoundError`. Also surfaces the internal "Kai" codename (Bug 18) and exposes the AI-assisted development workflow in source comments: → `tell Claude 'groq done'` and it will flip the status.

Deep Audit — Round 9 (Latest Build + Untested Commands)

Pulled latest via `curl -fsSL https://railcall.ai/install.sh | bash`. Tested previously untested commands (demo, rotate-key, backup, backup-verify) and probed new Studio API endpoints. Confirmed all open bugs still present. Root cause of CLI/Studio data divergence identified. Two new bugs found.

Round 9 Confirmed Working ✓

Test	Result
<code>railcall demo</code> — full golden path	✓ build → receipt → verify all pass
<code>railcall rotate-key</code>	✓ new key minted, old receipts still verify
<code>railcall backup</code>	✓ archive created, ed25519-signed
<code>railcall backup-verify</code>	✓ signature verified, 0 issues
Old receipts after key rotation	✓ SIGNATURE VALID (flat receipts carry their own key)
<code>/api/commands/list</code>	✓ 13 commands listed, wired status accurate

Round 9 Still Open ✗

Bug	Confirmed
Bug 25 — Python version in Server header	✗ Server: BaseHTTP/0.6 Python/3.14.4
Bug 30 — 4 commands unwired	✗ <code>github.create_issue</code> , <code>slack.post_message</code> , <code>stripe.read_balance</code> , <code>notion.export_session</code>
Bug 36 — doctor wrong model advice	✗ still warns <code>qwen2.5-coder:1.5b</code> , ignores auto-detected <code>qwen:7b</code>
Bug 40 — Studio receipts show demo stubs	✗ root cause confirmed (see Bug 44)
Bug 41 — internal files in workbench	✗ all files still present on latest build

Bug 43 — `/api/usage` serves stale balance with no TTL (Medium) — New

Actual response:

```
{"status": "CACHED", "flows": 90, "tier": "free", "checked_min_ago": 433}
```

433 minutes — over 7 hours old. When the gateway is unreachable the Studio falls back to a cached value with no expiry. A user could run their account to 0 flows while the Studio dashboard still shows 90. There is no staleness warning in the UI and no forced refresh after a threshold.

Bug 44 — CLI and Studio operate on entirely separate workspaces (High) — New

Root cause confirmed for Bug 40 and the Bug 28 partial fix. The Studio server defines:

```
HERE = os.path.dirname(os.path.abspath(__file__)) # ~/.railcall/station/workbench/
ROOT = os.path.dirname(HERE) # ~/.railcall/station/
WS = os.path.join(ROOT, ".railcall_workspace") # ~/.railcall/station/.railcall_workspace/
```

The CLI writes everything to ~/.railcall/:

Data	CLI path	Studio path
audit_log.jsonl	~/.railcall/audit_log.jsonl	~/.railcall/station/.railcall_workspace/audit_log.jsonl
receipts/	~/.railcall/receipts/	~/.railcall/station/.railcall_workspace/receipts/
monitor events	written by CLI	Studio reads from its own WS

These paths never share data. Every run in the CLI is invisible to the Studio. The Studio /api/monitor always shows 0 events. The Studio /api/receipts/list always shows the pre-installed demo stubs because the real receipts are in a different directory. This is the single root cause behind three previously reported bugs (27, 28, 40).

Security Probes — All Rounds

Probe	Result
Tamper detection — modify receipt field	✓ SIGNATURE INVALID
PII detection — email, phone, SSN columns	✓ Flagged correctly
Formula injection — =cmd ..., @SUM(...), -2+cmd	✓ Flagged (after fix)
Key vault permissions — keys.local.json, token.json	✓ chmod 600
Webhook 64KB limit	✓ Rejected cleanly
CSRF — no Origin, external Origin	✓ 403
Session gate /api/chat — no token	✓ 403
/api/save_key — no Origin	✓ 403
Approval token — /api/approve_token	✓ 404
Key injection — /api/keys, /api/integrations/connect	✓ 404
Path traversal — railcall audit ../../etc/passwd	✓ File not found
Historical receipt verify after key rotation	✓ SIGNATURE VALID
API key leakage in audit_log.jsonl	✓ Clean — only key_id
Tamper formula_injection_cells in signed receipt	✓ SIGNATURE INVALID
Supply-chain pin — 4 Python files	✓ Pinned and verified
Supply-chain pin — Studio bundle (22MB)	× No hash check
Daemon /interpret — unauthenticated call	× No auth required
Daemon /compile — unauthenticated receipt generation	× No auth required
Daemon /governed — fresh install	× Broken (missing file)
Studio /api/receipts/list — real vs demo receipts	× Still shows demo stubs
wire_groq.py execution on user machine	× FileNotFoundError

Bug Summary — All 44 Bugs

#	Bug	Severity	Status	Category
1	PATH not active after install	Medium	✅ Fixed	Install
2	Contradictory cryptography installer messages	Low	✅ Fixed	Install
3	cryptography silently fails on PEP 668 / Homebrew Python	High	✅ Fixed	Install / Trust
4	Hosted engine HTTP 502 on all workflow builds	High	✅ Fixed	Core Feature
5	Workflow Builder rejects plain-English local steps	Medium	Open	UX
6	No sign-out button in logged-in dashboard	Medium	✅ Fixed	Auth / UX
7	Sign In / Sign Up navbar styling inconsistency	Low	✅ Fixed	UI
8	railcall interpret HTTP 404 — hardcoded model	Medium	✅ Fixed	CLI
9	interpret generates Python with syntax errors	Medium	⚠️ Partial	CLI
10	railcall audit accepts non-CSV without warning	Low	✅ Fixed	CLI
11	railcall backup exits code 1 on empty workspace	Low	✅ Fixed	CLI
12	railcall audit reads system files without restriction	Medium	✅ Fixed	CLI / Security
13	Formula injection not detected by audit	Medium	✅ Fixed (our feat/formula-injection, merged)	CLI / Security
14	railcall build hardcoded to internal power-grid schema	Medium	✅ Fixed	CLI
15	MCP server filename mismatch with docs	Low	Open	Docs
16	Studio pages hard-wired to port 8099 not running	High	Open	Studio
17	"Metercall" former brand across 21 source files	Medium	Open	Branding

#	Bug	Severity	Status	Category
18	Internal codename "Kai" in shipped Studio bundle	Low	Open	Branding
19	TENANT_ID = "tenant_alpha" hardcoded	Low	Open	Studio
20	Every CLI run overwrites same receipt file	High	⚠ Partial (audit only)	CLI / Receipts
21	No CLI commands for receipt history	Medium	✅ Fixed	CLI
22	Dashboard shows wrong model as healthy	Medium	✅ Fixed	CLI
23	railcall studio doesn't open browser automatically	Medium	✅ Fixed (our PR #3, merged)	CLI
24	/api/chat no rate limiting — each call burns a flow	Medium	Open	Studio / API
25	Python version exposed in Server header	Low	Open	Studio
26	Internal docs, competitor corpus, attack tests in every install	Critical	⚠ Partial (see 35/41)	Install / InfoSec
27	Studio receipt center shows only demo receipts	High	⚠ Regression (Round 8)	Studio / Receipts
28	audit_log.jsonl never created	High	✅ Fixed	CLI / Audit
29	Studio bundle downloaded without SHA256 verification	Medium	Open	Install / Security
30	4 of 13 registered commands not wired	Medium	Open	Studio
31	railcall login saves bad key before verifying	High	✅ Fixed (our PR #3, merged)	CLI / Auth
32	railcall login "" accepts empty string	Medium	✅ Fixed (our PR #3, merged)	CLI / Auth
33	receipts suggested verify path fails	Medium	⌚ Pending Merge (PR #5)	CLI
34	Build and interpret receipts still overwrite	Medium	✅ Fixed	CLI / Receipts
35	P0 packaging leak still present (internal files in workbench/)	Critical	Open	Install / InfoSec
36	railcall doctor recommends wrong model	Medium	Open	CLI

#	Bug	Severity	Status	Category
37	Daemon /interpret uses hardcoded model, not auto-detected	Medium	Open	Daemon / CLI
38	Daemon /compile + /interpret require no auth — unsigned receipts forgeable	Medium	Open	Daemon / Security
39	/governed endpoint broken on all fresh installs	Medium	Open	Daemon
40	Bug 27 NOT fixed — Studio receipt center still shows demo stubs	High	Open	Studio / Receipts
41	Bug 35 expanded — 10+ additional internal files shipped	Critical	Open	Install / InfoSec
42	wire_groq.py hardcoded to internal dev paths	Low	Open	Install
43	/api/usage serves stale cache with no TTL — 7+ hours old	Medium	Open (new)	Studio / API
44	CLI and Studio use separate workspaces — audit log never shared	High	Open (new)	CLI / Studio

Fixed: 13 · Partial: 4 · Pending Merge: 1 · Open: 26



RailCall Builders Contest — Submission v2

Pre-Push Corrections / Reviewer Addendum · 2026-07-08

This addendum supersedes the inconsistent rows in the original 26-page PDF. It does not remove any finding; it clarifies scope, counts, and wording so the report is internally consistent before submission.

1. Missing Bug 35 section restored

Bug 35 — P0 packaging leak still present: internal workbench files ship in the installed Studio bundle (Critical) — Open.

Fresh-install inventory still shows internal planning/spec/test artifacts inside the shipped `~/.railcall/station/workbench/` tree. This is the parent packaging leak referenced by Bug 26 and expanded by Bug 41. The fix is a release-packaging allowlist: ship only runtime files required by Studio/CLI, exclude internal specs, corpora, adversarial tests, dev telemetry, and one-off wiring utilities.

2. Bug 20 status correction

The body states build/UI/flow/interpret receipts now persist to unique timestamped files. Therefore the summary row for Bug 20 should read **Fixed**, not **Partial (audit only)**. Updated aggregate count: **Fixed 14 · Partial 3 · Pending Merge 1 · Open 26**.

3. Bug 38 wording correction

The issue is not “unsigned receipts forgeable.” The tested issue is: **daemon /compile and /interpret are unauthenticated local endpoints that can generate signed receipts without CLI/session authorization**. Better category: local governance/auth bypass for signed receipt generation.

4. Bugs 16–19 scope clarification

These should be read as **source-tree / packaged-artifact hygiene findings unless reproduced in the shipped /v2 runtime**. The report already notes that the running Studio differs from the React source tree. If not visible in the running product, they should not be counted as active runtime UX failures.

5. Bug 41 sensitivity clarification

The Ed25519 public key hex is public by design and is not itself a secret. The actual severity is that **internal telemetry/dev audit state and internal test artifacts** ship to users. The fix remains the same: package/runtime allowlist plus generated-release inventory check.

Go / No-Go

With this addendum attached, the report is acceptable to share as a feedback/audit submission. Without it, the missing Bug 35 body section and Bug 20 status mismatch can make the report look internally inconsistent.

Secrets check: this addendum contains no API keys, tokens, passwords, private keys, or live credentials.